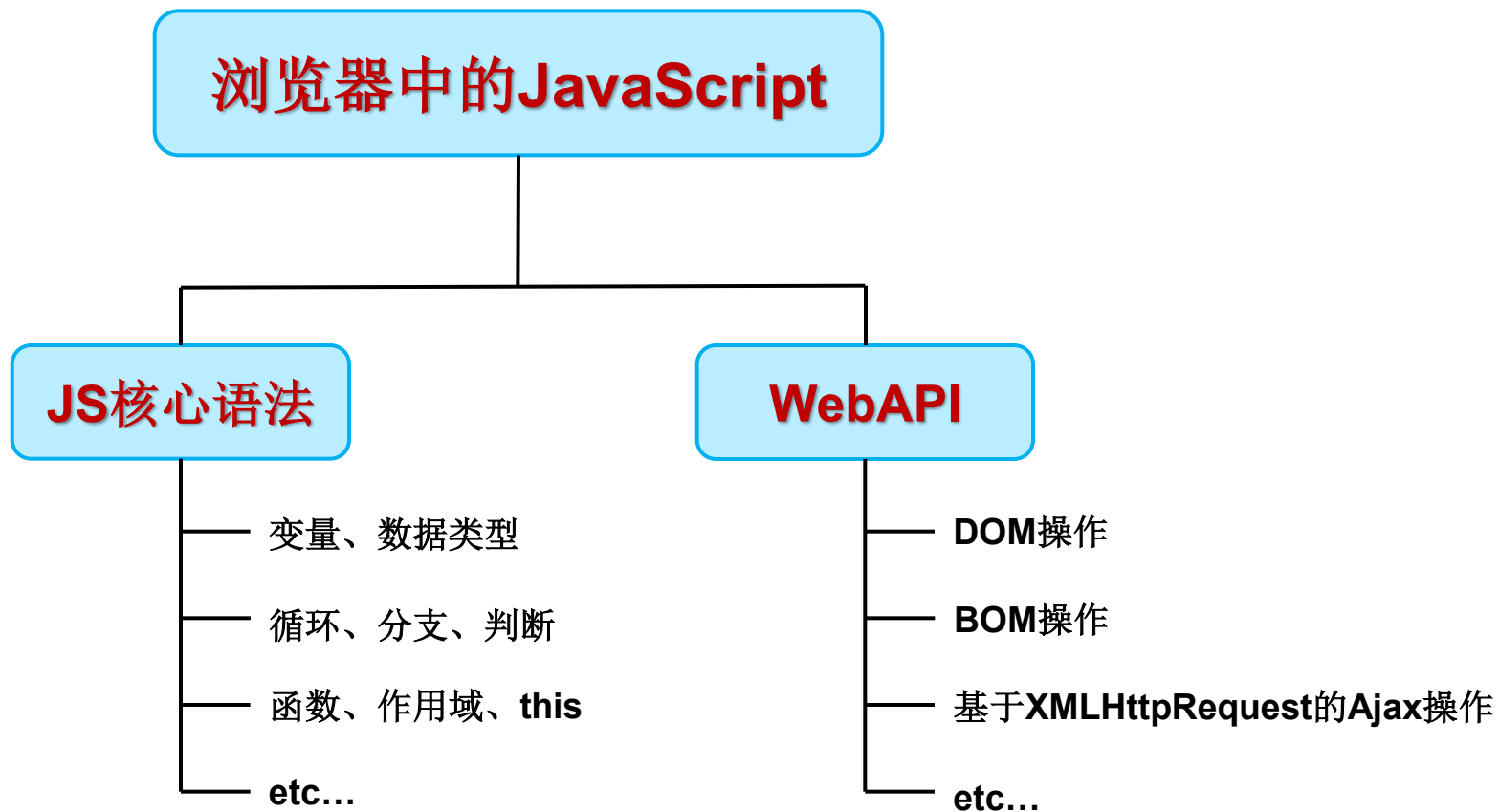


Node.js

一、浏览器中的JavaScript

1. 浏览器中的JavaScript组成部分



2.JavaScript解析引擎

浏览器

待执行的JS代码

JavaScript解析引擎

不同的浏览器使用不同的JavaScript解析引擎：

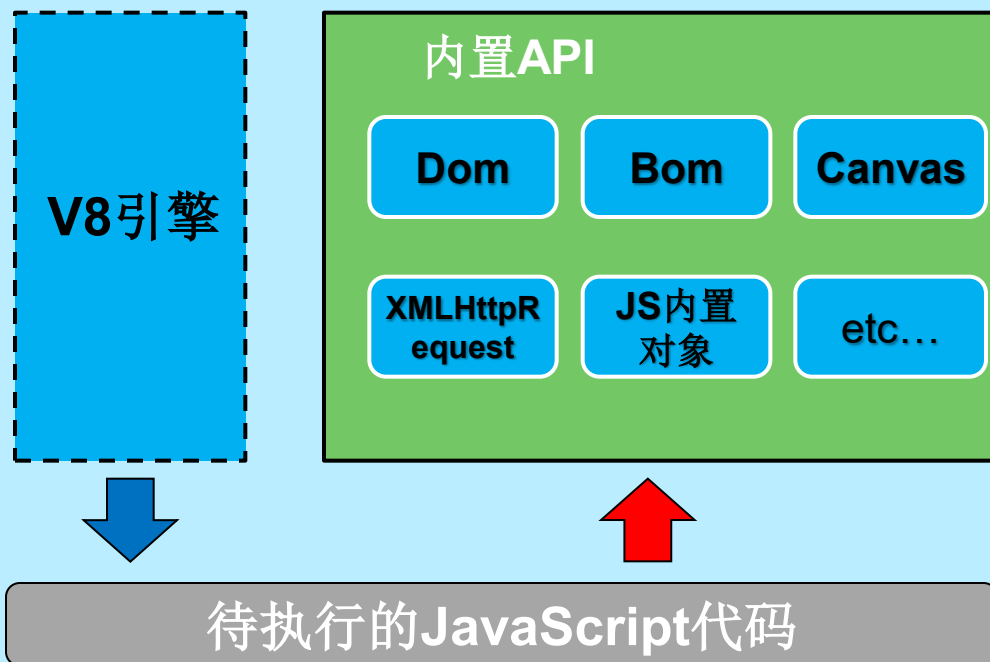
- Chrome浏览器 => V8
- Firefox浏览器 => OdinMonkey
- Safari浏览器 => JSCore
- IE浏览器 => Chakra
- etc..

其中，Chrome浏览器的V8解析引擎性能最好！

3.浏览器中的JavaScript运行环境

运行环境：是指代码正常运行所需的必要环境。

Chrome浏览器运行环境



总结：

①V8引擎负责解析和执行JavaScript代码。

②内置API是由运行环境提供的特殊接口，只能在所属的运行环境中被调用。

4.JavaScript能否做后端开发



Java



Python



PHP



Node.js

二、Node.js简介

1. 什么是Node. js?

- 简单的说 Node.js 就是运行在服务端的 JavaScript;
- Node.js是一个基于Chrome JavaScript 运行时建立的一个平台。

2. Node.js 特点

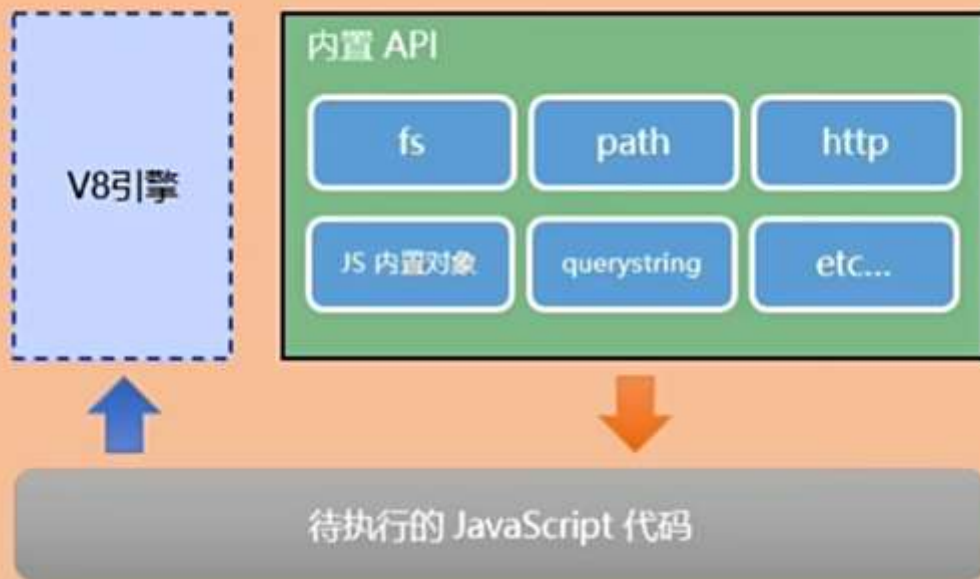
- **单线程和事件驱动架构：**Node.js 使用单线程来处理请求，但通过事件驱动和非阻塞 I/O 操作的特性，使其可以高效地处理大量并发连接，而不会阻塞线程；
- **异步和非阻塞 I/O：**这使得 Node.js 能够处理高并发的请求，非常适合 I/O 密集型应用，如文件读取、数据库操作和网络请求；
- **跨平台支持：**Node.js 可以在 Windows、Linux 和 macOS 等多个操作系统上运行；
- **庞大的生态系统：**Node.js 拥有一个名为 npm（Node Package Manager）的生态系统，其中有丰富的第三方库和模块，可以用来扩展功能和加速开发。

3. Node. js的应用场景

- **Web 服务器：**由于其高效的处理并发请求的能力，常被用于构建快速、可扩展的 web 服务器。
- **实时应用：**例如聊天应用、实时协作工具等。
- **API 服务：**Node.js 很适合用来开发 RESTful API 和 GraphQL 服务。
- **命令行工具：**Node.js 还可以用于构建 CLI（command-line interface）工具，利用其易用的库和包管理器进行快速开发。

4. Node.js 的运行环境

Node.js 运行环境



注意:

- ① 浏览器是 JavaScript 的前端运行环境。
- ② Node.js 是 JavaScript 的后端运行环境。
- ③ Node.js 中无法调用 DOM 和 BOM 等浏览器内置 API。

5. Node. js的安装

登陆网站:

<https://nodejs.org/en>

安装成功测试:

```
$ node -v
```

三、Node.js开发

1. Node.js 创建第一个应用

- 使用 Node.js 时，我们不仅仅在实现一个应用，同时还实现了整个 HTTP 服务器；
- Node.js 应用是由哪几部分组成的：
 - **require 指令**：在 Node.js 中，使用 **require** 指令来加载和引入模块，引入的模块可以是**内置模块**，也可以是**第三方模块**或**自定义模块**。
 - **创建服务器**：服务器可以监听客户端的请求，类似于 Apache、Nginx 等 HTTP 服务器。
 - **接收请求与响应请求**：

Node.js中模块的分类:

Node.js中根据模块来源的不同, 将模块分为了3大类, 分别是:

- 内置模块(内置模块是由Node.js官方提供的, 例如fs、path、http等);
- 自定义模块(用户创建的每个.js文件, 都是自定义模块);
- 第三方模块(由第三方开发出来的模块, 并非官方提供的内置模块, 也不是用户创建的自定义模块, 使用前需要先下载);

加载模块：

使用强大的 `require()` 方法，可以加载需要的内置模块、用户自定义模块、第三方模块进行使用。例如：

// 1. 加载内置的 `fs` 模块

```
const fs = require('fs')
```

// 2. 加载用户自定义模块

```
const custom = require('./custom.js')
```

// 3. 加载第三方模块

```
const moment = require('moment')
```

第三方模块（包）：

为何用？

由于Node. js的**内置模块**仅提供了一些底层的API，导致在基于内置模块进行项目开发时，效率很低。包是基于内置模块封装出来的，提供了更高级、更方便的API，极大的提高了开发效率。

包管理工具：

npm, Inc. 公司提供了一个包管理工具，我们可以使用这个包管理工具（NodePackageManager，简称npm），这个包管理工具随着Node.js的安装包一起被安装到了用户的电脑上。大家可以在终端中执行npm-v命令，来查看自己电脑上所安装的npm包管理工具的版本号。

<https://www.npmjs.com/>

其中：

- node_modules 文件夹用来存放所有已安装到项目中的包。require() 导入第三方包时，就是从这个目录中查找并加载包。

初次安装包后多了哪些文件：

初次安装包完成后，在项目文件夹下多一个叫做 **node_modules** 的文件夹和 **package-lock.json** 的配置文件。

- **package-lock.json** 配置文件用来记录 **node_modules** 目录下的每一个包的下载信息，例如包的名字、版本号、下载地址等。

2. Express

Express是基于Node.js平台，快速、开放、极简的Web开发框架。

通俗的理解：Express的作用和Node.js内置的http模块类似，是专门用来创建Web服务器的。

Express的本质：就是一个npm上的第三方包，提供了快速创建Web服务器的便捷方法。

Express的中文官网：

<http://www.expressjs.com.cn/>

进一步理解Express:

使用Node.js提供的原生http模块也可以创建Web服务器，但http内置模块用起来很复杂，开发效率低；**Express**是基于内置的http模块进一步封装出来的，能够极大的提高开发效率。

```
$ npm install express
```

Express的基本使用:

- 监听GET请求:

通过app.get()方法，可以监听客户端的GET请求，具体的语法格式如下：

```
app.get('请求URL', function(req, res){/*处理函数*/})
```

//参数1：客户端请求的URL地址

//参数2：请求对应的处理函数

// req：请求对象（包含了与请求相关的属性与方法）

// res：响应对象（包含了与响应相关的属性与方法）

- 监听POST请求

通过`app.post()`方法，可以监听客户端的POST请求，
具体的语法格式如下：

`app.post('请求URL', function(req, res){/*处理函数*/})`

//参数1： 客户端请求的URL地址

//参数2： 请求对应的处理函数

// req： 请求对象（包含了与请求相关的属性与方法）

// res： 响应对象（包含了与响应相关的属性与方法）

- 把内容响应给客户端:

通过`res.send()`方法，可以把处理好的内容，发送给客户端：

```
app.get('/user/', (req, res) => {  
  //向客户端发送JSON对象  
  res.send ({ "name": 'zs', "age": 20, "gender": '男' })  
})
```

```
app.post('/user', (req, res) => {  
  //向客户端发送文本内容  
  res.send('请求成功')  
})
```

上机实验三：留言板制作

上机指导书：

上机报告：

上机时间：

此题未设置答案，请点击右侧设置按钮

1.以下哪一项不是WebAPI?

- ☐ A BOM
- ☐ B DOM
- ☐ C JSON
- ☐ D XMLHttpRequest

提交

此题未设置答案，请点击右侧设置按钮

2.node.js基于以下哪一个JS的解析引擎？

- ☐ A JSCore
- ☐ B OdinMonkey
- ☐ C V8
- ☐ D Chakra

提交

此题未设置答案，请点击右侧设置按钮

3.不需要用require语句加载的第三方模块为（）？

- ☐ A express
- ☐ B mysql
- ☐ C body-parser
- ☐ D npm

提交

项目概述：

该程序使用 **HTML**、**CSS**、**JavaScript** 作为前端，**Node.js** 和 **Express** 作为后端，**MySQL** 作为数据库。

环境要求：

在开始安装之前，请确保您的系统满足以下要求：

- Node.js**
- npm** (**Node.js** 包管理器，通常随 **Node.js** 一起安装)
- Express**
- **MySQL**